

Short-Answers-Broken-RAG

Fine-tuning vs. retrieval - why the reader, not the retriever, is a small model's ceiling

1. What I built

I built three versions of a Yu-Gi-Oh question-answering system on Gemma 2 2B and compared them on the same held-out questions. System A is the untouched base model, closed-book; System B is my QLoRA fine-tune, also closed-book; System C is that fine-tune with a retriever feeding it passages from my own corpus. I chose Yu-Gi-Oh rulings and card facts on purpose - the base model clearly does not know them (about 3 out of 12 on a hand probe, confidently inventing card text) - so the three systems can actually separate. Every answer is graded by a reference-grounded AI judge against a written gold answer and its exact evidence.

2. Headline result - two evaluations (mean score / 10, n = 60 each)

System	Biased 60	Balanced 60
 A base, closed-book	3.98	1.93
 B fine-tune, closed-book	5.25	2.62
 C fine-tune + retrieval	8.05	8.25

I evaluated twice, because my first 60 questions were skewed (45% rulings, 45% card facts): once on that original set, and again on a category-balanced 'unbiased' 60 (15 per category). On the balanced set the closed-book systems collapse while **C holds up**, so retrieval's advantage is larger once the bias is removed. Paired significance (bootstrap CI + t-test + Wilcoxon): B beats A strongly on the biased set (+1.27, p = 0.007) but only marginally on the balanced one (+0.68, p = 0.04); retrieval (B -> C) is strongly significant on both.

3. The RAG study - is retrieval or the reader the bottleneck?

System C stalls near **8/10** on answer quality, even though retrieval already fetches the right passage **93%** of the time. These are two different measures - 93% is a retrieval number (is the right passage found?), while 8/10 is the judge's answer-quality score - so the gap between them is itself the clue: the passage is usually there, but the answer still is not a 10. That is why I pointed my experiments at the remaining 7% - the questions where the RAG could be blamed - to see whether fixing retrieval would fix the score. Every retrieval-side experiment failed to move it, which is what indirectly proved the problem is the reader, not my RAG implementation. What each showed:

- **Reranking** (re-order the retrieved passages): no help - slightly worse, 8.05 -> 7.55.
- **Deeper retrieval** (pull more passages): recall rose 0.93 -> 0.97, but the score did not move.
- **Repairing a split-effect chunk**: I reconstructed a card's missing effect text back into the context - and the model still dropped it.
- **Six cheap reader fixes** (self-consistency, self-checking, quote-then-answer): all failed.
- **A stronger reader on the same context**: the only thing that worked - given the identical passages, a stronger model produced the complete, correct answer where my fine-tune under-answered.

So even after aiming squarely at the 7%, nothing on the retrieval side moved the number - the limit is my 2.6-billion model reading what it already has.

A few things the debugging pinned down:

- The reranker was not broken - it kept the gold passage at rank 1 (recall@5 stayed 0.93) and only reshuffled the surrounding passages, which is enough to wobble a small model's answer.
- Two-thirds of the lost points are 'card-fact' questions, but reading them showed the model inverting yes/no rulings it had the passage for - a reading failure, not a missing fact.
- The one real chunking bug I found (an effect split across two passages) I repaired - and the model still dropped the recovered clause: proof the context was already enough.

4. What I concluded

The bottleneck is not the retriever, it is the reader. Fine-tuning teaches the **shape** of an answer and retrieval supplies the **facts** - correctness only jumps once real passages are in the prompt - but a small model mis-reads passages it already holds, and no retrieval trick fixes that. Retrieval engineering is a dead end once recall is high; only a stronger reader, or retraining the model to read from its context, moves the number. I confirmed this with a controlled test: on the same passages, a stronger model produced complete, correct answers where my 2.6B reader did not.

5. What surprised me

My fine-tune's own gain shrank to marginal once I balanced the categories - a good reminder that my earlier, skewed test set had been quietly flattering it, and that I should check the make-up of an evaluation set before trusting a headline number.

6. What I learnt

Three lessons stuck with me. First, **test the cheap thing before you assume** - the small experiments (reranking, deeper retrieval, chunk-repair) told me where the problem was not, and saved me from paying to fix the wrong thing. Second, **a skewed test set quietly flatters your model** - my fine-tune looked clearly better until I rebalanced the categories and its edge shrank to marginal, so I now check the make-up of an evaluation set before I trust a number. Third, **a reference-grounded judge lets me score a domain I do not know** - handing it a gold answer and its evidence gave me reproducible marks on Yu-Gi-Oh without my being an expert.

7. An open question

One thing I could not fully test, and I would put it to anyone reading this: my fine-tuning answers had a median length of only 17 words. Could that short-answer habit be leaking into System C, making it drop details even when the passage clearly holds them - a better explanation for the incomplete answers than anything on the retrieval side? I would welcome other views on it.

External evidence for the reader bottleneck. Pandey (2026), "Can Small Language Models Use What They Retrieve?", arXiv:2603.11513 - even a 7B model uses an oracle-retrieved passage only ~15% of the time. Baturava et al. (2026), "Little Brains, Big Feats", arXiv:2606.30062 - on fixed context, the reader alone swings answer quality. Liu et al. (2025), "ROSE-RAG", Findings of ACL - prompt-only fixes fail on small readers.